

# Point.ai

AI, human review, and CAD automation

SYSTEM ARCHITECTURE AND OPERATIONS MANUAL

31 DAYS

INVESTED

**STACK:** Python, FastAPI, Uvicorn, Pydantic, OpenCV, NumPy, SciPy, scikit-image, Shapely, PyTorch, torchvision, timm, segmentation-models-pytorch, Hugging Face Hub, ezdxf, React, TypeScript, Vite, Tailwind CSS, Three.js, Supabase, Docker, Railway, Vercel, AutoCAD, AutoLISP, MCP

# Contents

---

|    |                                                        |
|----|--------------------------------------------------------|
| 01 | 1. System charter                                      |
| 02 | 2. Architecture                                        |
| 03 | 3. Data structures and persistence model               |
| 04 | 4. End-to-end operational workflow                     |
| 05 | 5. AI and inference subsystem                          |
| 06 | 6. Geometry and CAD subsystem                          |
| 07 | 7. Human-in-the-loop controls                          |
| 08 | 8. Frontend, API, persistence, and deployment surfaces |
| 09 | 9. Configuration and operational safeguards            |
| 10 | 10. Capability and status matrix                       |
| 11 | 11. Strategic structured-CAD blocker                   |
| 12 | 12. Product direction and FloorplanFit                 |
| 13 | 13. Failure modes and safeguards                       |
| 14 | 14. Extension strategy                                 |
| 15 | 15. Technology stack                                   |
| 16 | 16. Operator checklist                                 |

# 1. System charter

## 1.1 Product mission

Point.ai converts a residential floor-plan image into reviewable geometric data and an editable DXF suitable for downstream CAD work.

The system is designed around a controlled transformation:

```
raster floor plan
-> inferred structural evidence
-> normalized geometry
-> deterministic cleanup and validation
-> human review of ambiguous openings
-> DXF generation
```

The product is not a replacement for AutoCAD and does not treat a visually plausible preview as proof of construction-grade geometry. Its responsibility is to produce an auditable CAD candidate, expose uncertainty, and preserve a review path before the result is treated as authoritative.

## 1.2 Product boundary

Point.ai is responsible for:

- accepting an image, an existing geometric structure, or a legacy room-based plan;
- selecting and running an inference backend;
- normalizing walls, openings, junctions, scale metadata, and quality signals;
- generating structural or mask-region DXF output;
- exposing previews, artifacts, and review flags;
- allowing a user to move, orient, or remove proposed doors and windows;
- persisting projects, plans, reviewed openings, and optional scene state.

Point.ai does not currently guarantee:

- construction-authoritative reconstruction from arbitrary raster images;
- complete recovery of every missing wall, door, window, room, label, or dimension;
- a fully typed parametric building model;
- a general-purpose topology solver;
- a browser-based CAD replacement;
- a general site-fit solver in the canonical application;
- active 3D product behavior, even though 3D code and assets are present.

## 1.3 Authority model

The system uses multiple levels of authority:

| Level                        | Role                                                          |
|------------------------------|---------------------------------------------------------------|
| Source image                 | Visual reference and inference input                          |
| Model output                 | Probabilistic proposal, never final authority                 |
| Normalized structure         | Deterministic geometric candidate used by the active pipeline |
| Reviewed opening annotations | Human-approved override for door/window generation            |
| Generated DXF                | Operational CAD output, subject to quality review             |

| Level                | Role                                     |
|----------------------|------------------------------------------|
| Persisted plan state | Reusable application state for a project |

The model is an assistant. Human-reviewed state and deterministic geometry are stronger than raw inference, but neither should be confused with a fully validated construction document.

## 2. Architecture

### 2.1 System context

```
Browser application
-> FastAPI
-> input resolver
-> legacy plan adapter
-> existing structure parser
-> image inference router
-> geometric post-processing
-> quality gate
-> DXF generation
-> artifact storage
-> Supabase

Optional inference worker
<- validated worker contract

Optional CAD automation
<- ezdxf headless path
<- AutoCAD MCP path
```

### 2.2 Subsystem responsibilities

| Subsystem                | Primary responsibility                                            |
|--------------------------|-------------------------------------------------------------------|
| React frontend           | Upload, project navigation, previews, review, save, download      |
| FastAPI routes           | HTTP boundary, response contracts, error mapping                  |
| Parse service            | Resolve plan, structure, or image input                           |
| Inference router         | Select local, ensemble, experimental, or remote inference         |
| Plan parser              | Normalize public geometry and adapt legacy room plans             |
| Structure post-processor | Snap, merge, filter, classify, build junctions, anchor openings   |
| Quality gate             | Convert structural anomalies into metrics and review reasons      |
| DXF service              | Select structural or mask-region generation and build previews    |
| Artifact service         | Persist per-run structures, metrics, previews, and debug payloads |
| Supabase integration     | Authenticate users and persist project/plan state                 |
| Worker contract          | Validate remote wall/opening detections                           |
| CAD automation           | Read, write, render, inspect, or automate DXF/AutoCAD operations  |

### 2.3 Logical layering

Point.ai is organized as a full-stack application with the following practical layers:

- **Presentation** - React components and feature modules.
- **API contracts** - Pydantic request/response envelopes and TypeScript response types.
- **Application services** - parsing and DXF-generation orchestration.
- **Geometry domain** - walls, openings, junctions, dimensions, measurement, and quality rules.

- **AI infrastructure** - model loading, local inference, ensemble coordination, and remote workers.
- **CAD infrastructure** - ezdxf, templates, rendering, and AutoCAD automation.
- **Persistence** - Supabase authentication and PostgreSQL JSONB state.
- **Observability** - structured logs, run artifacts, quality metrics, and provenance payloads.

The architecture should keep these responsibilities separate. HTTP routes must not absorb inference logic, geometry mutation, persistence, or CAD rendering.

## 2.4 Boundary ownership

Every subsystem owns a specific kind of truth:

- the frontend owns transient interaction state and user intent;
- API envelopes own transport validation;
- the parser owns normalization into the public structure;
- the post-processor owns deterministic cleanup decisions;
- the quality gate owns review classification, not geometry repair;
- the DXF service owns output-mode selection and file generation;
- Supabase owns durable user/project/plan state;
- the artifact service owns diagnostic run outputs;
- external workers own computation only and must return contract-valid evidence.

Crossing these boundaries weakens auditability. In particular, model-specific payloads should not leak directly into CAD generation, and UI-specific annotations should not become canonical geometry without explicit translation.

---

# 3. Data structures and persistence model

## 3.1 API envelopes

The API uses Pydantic models for its outer boundary.

ParseStructureRequest accepts:

- plan: optional legacy room-based plan;
- structure: optional wall/opening structure;
- image: optional base64 image;
- scale\_hint: optional coordinate calibration;
- model\_variant: optional inference selection.

GenerateStructureRequest adds:

- annotations: optional reviewed opening segments.

The parse response returns:

- normalized structure;
- preview and artifact URLs;
- quality\_metrics;
- needs\_review;
- review\_flags;
- auto\_annotations.

The generation response also returns:

- `dxurl`;
- `scale_status`.

Pydantic validates the envelope. The nested geometry remains a dictionary-shaped contract normalized by application code rather than a complete hierarchy of nested Pydantic domain models.

## 3.2 Canonical geometric structure

The active public structure is:

```
{
  "model": "plan or model name",
  "source": "adapter or inference source",
  "walls": [],
  "openings": [],
  "junctions": [],
  "structure_meta": {},
  "pipeline_debug": {}
}
```

### Wall

```
{
  "id": "wall-0001",
  "orientation": "horizontal",
  "polyline": [
    { "x": 120.0, "y": 80.0 },
    { "x": 420.0, "y": 80.0 }
  ],
  "thickness": 6.0,
  "is_exterior": true,
  "confidence": 0.94,
  "side": "bottom"
}
```

The active parser expects a two-point, axis-aligned wall. Region-based MitUNet generation can maintain polygonal wall evidence separately under metadata and debug artifacts.

### Opening

```
{
  "id": "opening-0001",
  "kind": "door",
  "wall_id": "wall-0001",
  "position": { "x": 230.0, "y": 80.0 },
  "offset": 92.0,
  "span": 36.0,
  "orientation": "horizontal",
  "side": "bottom",
  "confidence": 0.88,
  "swing": "up",
  "door_type": "normal"
}
```

An opening must have a valid kind, positive span, and sufficient placement evidence. The post-processor attempts to select a compatible host wall and rejects candidates that cannot fit safely.

### Junction

```
{
  "point": { "x": 420.0, "y": 80.0 },
  "type": "T",
  "wall_ids": ["wall-0001", "wall-0007"]
}
```

Supported junction categories are:

- L: endpoint-to-endpoint corner;
- T: one wall endpoint meets another wall body;

- X: two wall bodies cross.

This is a junction list, not yet a complete typed planar wall graph.

## Rooms

Rooms are supported mainly through the legacy input adapter. A legacy room provides rectangular geometry and room-level doors/windows. The adapter derives canonical walls and openings, then records room counts in quality metrics.

The current normalized output does not preserve rich rooms as first-class entities. Polygonal rooms, ownership, adjacency, mutability, and room minimums belong to richer catalog/site-fit concepts rather than the canonical active image-to-DXF structure.

## 3.3 Metadata, metrics, and review state

structure\_meta can contain:

- source image dimensions;
- unit, commonly pixel or inch;
- scale\_status;
- optional scale\_hint;
- effective DXF mode;
- opening-detection policy;
- reviewed-opening persistence marker;
- region-plan data;
- MitUNet debug data;
- provenance data.

quality\_metrics can contain:

- parser and source-contract identity;
- wall/opening/door/window counts;
- exterior/interior wall counts;
- raw, snapped, and merged wall counts;
- anchored and filtered opening counts;
- L/T/X junction counts;
- shell-coverage values;
- inference backend and model variant;
- DXF mode and region counts;
- quality-gate result and reasons.

review\_flags contain human-readable explanations of structural uncertainty. needs\_review is true when unresolved flags remain.

## 3.4 Opening review annotations

The frontend uses segment annotations:

```
interface OpeningAnnotation {
  type: 'door' | 'window'
  x1: number
  y1: number
  x2: number
  y2: number
  swing?: 'up' | 'down' | 'left' | 'right'
```

```
_source?: string
}
```

Before persistence, transient `_source` metadata is removed. A structure metadata marker records that reviewed annotations were saved.

### 3.5 Remote worker contract

The worker boundary validates:

- model identity and version;
- wall detections;
- opening detections;
- image dimensions;
- inference metadata;
- optional debug overlay.

Wall validation requires a positive thickness and exactly two polyline points. Opening validation requires a door/window kind and positive span. Invalid remote payloads are rejected with typed worker errors.

### 3.6 Supabase entities

```
authenticated user
1 -> many projects
project
1 -> many plans
```

projects

- UUID identifier;
- authenticated owner identifier;
- name;
- creation and update metadata.

plans

- UUID identifier;
- parent project identifier;
- name;
- source image data or storage reference;
- parsed structure JSONB;
- reviewed opening annotations JSONB;
- placed 3D items JSONB;
- floor material;
- wall material;
- creation and update metadata.

Row Level Security restricts project access to the owner and plan access through ownership of the parent project. Cascade deletion preserves relationship integrity.

### 3.7 Run artifacts

Each parse/generation request receives an isolated run directory in temporary storage. Available artifacts include:

- normalized structure JSON;



- quality JSON;
- main preview;
- structure preview;
- worker overlay;
- DXF region plan;
- MitUNet region debug;
- provenance payload.

Temporary artifact storage is operational, not durable. Durable product state belongs in Supabase or another explicitly managed object store.

---

## 4. End-to-end operational workflow

### 4.1 Input selection

The client submits exactly one practical source:

- a legacy plan;
- an existing structure;
- a base64 image.

If no supported source is provided, the API returns a validation error.

### 4.2 Parse and inference

For a legacy plan, the parser derives walls and hosted openings from room rectangles.

For an existing structure, the parser normalizes walls/openings, applies scale when provided, and runs geometric post-processing.

For an image:

- the parse service resolves the effective backend;
- inference returns raw walls, openings, masks, or annotations;
- the parser converts output into the public structure;
- model/backend identity is added to quality metrics.

### 4.3 Geometry processing

The deterministic pipeline:

- projects near-axis diagonal noise where applicable;
- normalizes wall geometry;
- snaps wall axes and endpoints;
- snaps candidate intersections;
- merges collinear spans;
- builds L/T/X junctions;
- classifies walls;
- removes text-like and isolated artifacts;
- filters furniture-like or duplicate openings;
- anchors valid openings to host walls;

- produces metrics, flags, and debug stages.

## 4.4 Quality gate

The quality gate evaluates:

- whether walls exist;
- whether exterior-wall evidence is plausible;
- whether the footprint covers enough bounding-box sides;
- whether openings are absent when opening detection is expected;
- whether previous stages raised review conditions.

The gate is a triage mechanism. It is not a proof of global topology or CAD correctness.

## 4.5 DXF mode resolution

The system chooses:

- `structural` for normalized wall/opening geometry;
- `mask_regions` when the effective MitUNet/ensemble path provides a wall mask suitable for region generation.

Reviewed annotations replace automatic opening proposals for the generation request.

## 4.6 Output

The service:

- writes the DXF to isolated temporary storage;
- attempts to build a DXF preview;
- saves run artifacts;
- returns the DXF URL, preview, structure, quality signals, review flags, and annotations.

## 4.7 Operational completion criteria

A request is technically complete when:

- the selected backend returned or a documented fallback was used;
- the structure passed transport and normalization rules;
- scale state is explicit;
- geometry stages produced a serializable structure;
- the quality gate produced reasons, even if the result requires review;
- the DXF writer completed;
- artifact and download URLs refer to the same request;
- reviewed annotations, when supplied, were applied instead of automatic proposals.

Completion does not mean professional acceptance. Professional acceptance requires inspecting the review signals and opening the generated file in the target CAD workflow.

## 5. AI and inference subsystem

### 5.1 Backend selection

The inference router supports:

| Backend         | Role                                                |
|-----------------|-----------------------------------------------------|
| heuristic_local | Lightweight OpenCV fallback                         |
| cubicasa_local  | Multi-task rooms/icons/openings inference           |
| mitunet_local   | Wall-mask inference                                 |
| ensemble_local  | MitUNet walls plus CubiCasa opening proposals       |
| r2v_local       | Experimental raster-to-vector path                  |
| remote          | External worker using the validated worker contract |

POINTAI\_INFERENCE\_BACKEND is the strongest backend override. Without an override, the router prefers CubiCasa when available and otherwise uses the heuristic backend. The container configuration selects the ensemble path.

### 5.2 Heuristic backend

The heuristic path uses image binarization, morphology, line detection, and approximate wall reconstruction. It is valuable as a low-dependency fallback and for image styles that cause learned models to underperform.

Its result should be treated as lower-confidence structural evidence.

### 5.3 CubiCasa

CubiCasa produces heatmaps, room classes, icons, and opening evidence. It is useful for door/window proposals but is sensitive to drawing style.

When CubiCasa returns too few walls, the router can fall back to the heuristic backend. Non-ensemble paths intentionally disable automatic opening detections in the finalized result, preventing weak opening output from silently becoming production geometry.

### 5.4 MitUNet

MitUNet is the main learned wall-mask source. It applies model inference, mask cleanup, region extraction, and region-based DXF preparation.

Its mask is evidence of wall footprint, not an authoritative wall graph.

### 5.5 Ensemble

The ensemble runs MitUNet and CubiCasa together:

- MitUNet proposes walls;
- CubiCasa proposes openings;
- openings are reanchored against MitUNet walls;
- missing side/swing semantics can be filled conservatively;
- results become editable image-coordinate annotations.

The structural openings list remains empty in this annotation-first path. Opening counts are promoted into quality metrics from annotations so the quality gate does not produce a false absence.

## 5.6 Remote worker

The remote path is appropriate when model inference must be separated from the API process. The client performs health checks, handles connectivity and timeout errors, validates payload shape, and converts worker detections into the canonical structure contract.

The worker must never be trusted without validation.

---

# 6. Geometry and CAD subsystem

## 6.1 Structural geometry

The structural generator consumes normalized walls and openings. It applies wall thickness rules, junction handling, opening cuts, door/window drawing, layers, and template conventions.

The active framing rule uses thicker exterior walls and thinner interior walls. Geometry must preserve units before those rules are interpreted as real-world dimensions.

## 6.2 Region geometry

The active mask-region path converts wall-mask evidence into editable rectangular regions:

```
wall mask
-> morphological cleanup
-> horizontal and vertical rectangle extraction
-> trimming and boundary clamping
-> rectangular wall regions
-> closed LWPOLYLINE and hatch output
```

This path can dramatically reduce entity count and improve editability, but rectangular regions do not establish semantic wall identity, room ownership, or correct global topology.

## 6.3 CAD output

DXF generation uses ezdxf and supports:

- layers;
- wall entities;
- door slabs and swings;
- window geometry;
- hatches;
- labels and dimensions;
- title-block/template content;
- previews.

AutoCAD automation is available through an MCP integration with a Windows/AutoLISP file-IPC path and a headless ezdxf path. These are CAD control surfaces, not substitutes for geometry validation.

## 6.4 Scale

Without calibration, image-derived geometry remains in pixels and `scale_status` remains unverified. A `scale_hint` converts coordinates and thicknesses into inches and marks the structure calibrated.

No downstream consumer should interpret pixel values as construction dimensions.

## 7. Human-in-the-loop controls

---

### 7.1 Review scope

The active review surface focuses on openings because doors and windows combine visual ambiguity with strict wall-hosting requirements.

The user can:

- select a proposed opening;
- move it;
- choose its direction;
- remove it;
- regenerate the DXF;
- save the reviewed annotations.

### 7.2 Control boundary

Reviewed annotations are request-level authority for opening generation. They override automatic annotations but do not rewrite the entire canonical structure.

The active editor cannot create a missing opening. It is therefore effective for false positives and placement correction, but incomplete for false negatives.

### 7.3 3D surface

The codebase contains Three.js walls, floors, openings, furniture, materials, lighting, walk mode, thumbnails, and scene persistence. The product flag disables the 3D experience.

Operational documentation and deployments should treat 3D as dormant unless the flag and its performance, persistence, and asset-loading implications are explicitly revalidated.

---

## 8. Frontend, API, persistence, and deployment surfaces

---

### 8.1 Frontend

The React/Vite application provides:

- authentication;
- project and plan navigation;
- image upload;
- generation status;
- previews;
- opening review;
- raw structure inspection;
- save and autosave;
- DXF download.

VITE\_API\_URL selects the backend. VITE\_SUPABASE\_URL and VITE\_SUPABASE\_ANON\_KEY enable persistence; without them, persistence is disabled and the application can operate in a direct mode.

## 8.2 API

Primary routes:

| Route                        | Function                                           |
|------------------------------|----------------------------------------------------|
| POST /api/v2/parse-structure | Parse/infer and return structure plus artifacts    |
| POST /api/v2/generate-dxf    | Parse/infer, apply reviewed openings, generate DXF |
| GET /downloads/{filename}    | Return generated DXF                               |
| /artifacts/...               | Serve per-run artifacts                            |

Worker failures are mapped to gateway-style errors. Input validation errors are mapped separately from generation failures.

## 8.3 Deployment

The backend container uses Python, Uvicorn, OpenCV, CPU PyTorch, ezdxf, and downloaded model weights. Railway is configured to build from the Dockerfile and restart after failures.

The frontend deploys as a Vite static application through Vercel.

## 8.4 Persistence

Supabase provides:

- email/password and Google authentication;
- PostgreSQL project/plan records;
- JSONB for structures, reviewed openings, and scene state;
- Row Level Security.

Large images or artifacts should move to managed object storage when direct JSON/text storage becomes operationally expensive.

# 9. Configuration and operational safeguards

## 9.1 Core configuration

| Variable                  | Purpose                           |
|---------------------------|-----------------------------------|
| POINTAI_INFERENCE_BACKEND | Force effective inference backend |
| POINTAI_WORKER_URL        | Remote worker base URL            |
| POINTAI_MITUNET_WEIGHTS   | MitUNet weight path               |
| POINTAI_CUBICASA_ROOT     | CubiCasa runtime root             |
| POINTAI_CUBICASA_MAX_SIDE | CubiCasa input-size ceiling       |
| POINTAI_CUBI_MIN_WALLS    | CubiCasa fallback threshold       |
| HF_TOKEN                  | Model-weight download access      |
| VITE_API_URL              | Frontend API endpoint             |
| VITE_SUPABASE_URL         | Supabase project URL              |
| VITE_SUPABASE_ANON_KEY    | Supabase public client key        |

Worker deployments also expose model identity, backend, weight, and readiness variables.

## 9.2 Required safeguards

- Treat all model output as untrusted structured input.
- Preserve worker validation at the network boundary.
- Reject unsupported backends explicitly.
- Keep scale status visible in every generated result.
- Preserve review flags; do not hide them to improve apparent success rate.
- Keep reviewed opening overrides distinct from automatic proposals.
- Use unique generated filenames and restrict download access to generated artifacts.
- Keep Row Level Security enabled.
- Do not log image payloads, tokens, or authentication secrets.
- Use durable storage for artifacts that must survive process or host replacement.
- Validate templates and model weights before declaring a backend ready.

## 9.3 Known configuration gap

The environment template documents configurable CORS origins, but the active API currently allows all origins directly in application code. Production hardening requires wiring the allowlist into the actual middleware configuration rather than relying on the unused documented variable.

## 9.4 Readiness checks

Backend readiness should be evaluated per capability rather than through process availability alone:

| Check             | Ready condition                                      |
|-------------------|------------------------------------------------------|
| API process       | Root endpoint responds                               |
| MitUNet           | Weight path resolves and model loads                 |
| CubiCasa          | Runtime root, imports, and selected weights resolve  |
| Ensemble          | Both MitUNet and CubiCasa are ready                  |
| Remote worker     | Health contract is valid and ready is true           |
| Structural DXF    | Required template/fallback and ezdxs are available   |
| Region DXF        | Wall mask and region metadata are available          |
| Persistence       | Supabase URL/key are configured and session is valid |
| Durable artifacts | Managed storage destination is configured            |

A healthy API process can still have unavailable model or persistence capabilities. Operators should expose capability readiness separately from basic service health.

## 10. Capability and status matrix

| Capability                               | State                    | Operational meaning                                 |
|------------------------------------------|--------------------------|-----------------------------------------------------|
| Image upload and parse                   | Implemented              | Accepts base64 image input                          |
| Existing structure parse                 | Implemented              | Normalizes wall/opening input                       |
| Legacy room-plan adapter                 | Implemented              | Converts room rectangles to geometry                |
| Heuristic inference                      | Implemented              | Lightweight fallback                                |
| CubiCasa inference                       | Implemented              | Optional learned backend                            |
| MitUNet wall inference                   | Implemented              | Main wall-mask backend                              |
| Ensemble inference                       | Implemented              | Wall mask plus reviewed opening proposals           |
| Remote worker                            | Implemented              | Contract-validated external inference               |
| Raster-to-Vector backend                 | Experimental             | Not production authority                            |
| Wall snapping/merging/filtering          | Implemented              | Deterministic cleanup                               |
| L/T/X junction list                      | Implemented              | Partial topology representation                     |
| Typed planar wall graph                  | Required                 | Needed for stronger CAD guarantees                  |
| Opening anchoring                        | Partial                  | Heuristic/annotation-first, not graph-authoritative |
| Quality gate                             | Partial                  | Triage, not full CAD validation                     |
| Structural DXF                           | Implemented              | Generates from normalized geometry                  |
| Region DXF                               | Implemented              | Generates from wall-mask regions                    |
| Opening review                           | Implemented              | Move, orient, delete                                |
| Missing-opening creation                 | Required                 | Needed to address false negatives                   |
| Supabase persistence                     | Implemented/configurable | Requires frontend environment                       |
| Per-run artifacts                        | Implemented              | Temporary filesystem storage                        |
| 3D editor                                | Implemented but disabled | Not an active product surface                       |
| General site-fit solver                  | Required elsewhere       | Not part of canonical Point.ai flow                 |
| Construction-authoritative raster-to-CAD | Required                 | Not guaranteed                                      |

## 11. Strategic structured-CAD blocker

### 11.1 Why segmentation is insufficient

A segmentation mask can indicate likely wall pixels. It cannot guarantee:

- exact endpoints;
- global continuity;
- closed room loops;
- shared junction identity;
- consistent wall thickness;
- exterior-shell correctness;
- opening ownership;



- opening offset and span;
- editable wall faces and joins;
- valid CAD topology.

Local visual evidence is especially weak for short branches, partially occluded walls, dimension-heavy drawings, diagonals, and intersections. A locally plausible repair can still break the global plan.

## 11.2 Required production-oriented architecture

The production path requires:

- **Typed wall graph**
  - nodes for corners and junctions;
  - edges for wall spans;
  - connected components;
  - exterior-shell identity;
  - wall thickness and support evidence.
- **Separate wall and opening models**
  - wall candidates from structural evidence;
  - doors/windows conditioned on existing wall candidates;
  - explicit host edge, offset, span, side, and swing.
- **Global geometric/topological solver**
  - exact endpoint coincidence;
  - loop closure;
  - gap and overshoot control;
  - collinear merging;
  - junction consistency;
  - width consistency;
  - opening-host constraints;
  - preference for the simplest valid graph.
- **Exact DXF renderer**
  - wall faces rather than only centerline patches;
  - resolved L/T/X joins;
  - trims, miters, and caps;
  - opening cuts;
  - stable layers and editable entities.
- **CAD validator**
  - topology checks;
  - shell closure;
  - connected-component checks;
  - opening attachment;
  - wall-width error;
  - reverse rasterization;
  - successful CAD opening and editing.

- **Representative benchmark**
- real Pointe-style plans;
- canonical expected walls, junctions, openings, and graphs;
- acceptance thresholds based on CAD structure, not only mask overlap.

### 11.3 Data requirement

A production-oriented proprietary path requires roughly **200-500 gold vector plans** with corrected:

- wall endpoints or edges;
- junctions;
- widths;
- exterior-shell membership;
- wall-attached openings;
- ownership relationships where required;
- verified CAD output.

Wall masks alone are not sufficient training truth for this contract.

## 12. Product direction and FloorplanFit

Point.ai’s concepts remain valuable:

- CAD extraction and automation;
- explicit artifacts;
- human review;
- geometry normalization;
- topology and ownership as first-class concerns;
- deterministic mutation;
- auditable output.

The favored product direction moves the geometry authority to FloorplanFit:

```
real DXF geometry
-> automatic extraction proposal
-> human-reviewed reusable curation
-> persisted canonical version
-> deterministic authorized changes
-> auditable export
```

This changes the risk boundary.

| Point.ai raster-first responsibility | FloorplanFit authority model                  |
|--------------------------------------|-----------------------------------------------|
| Infer geometry from pixels           | Read existing vector CAD geometry             |
| Guess scale and endpoints            | Normalize authoritative units and coordinates |
| Reconstruct topology                 | Derive and curate topology from CAD evidence  |
| Let model output drive geometry      | Let reviewed curation become canonical        |
| Repair the whole drawing implicitly  | Modify only explicitly authorized zones       |
| Depend on AI for geometry            | Keep AI optional and advisory                 |

FloorplanFit is not purely manual. Automatic extraction proposes artifacts; the user curates them; deterministic logic applies authorized changes. AI can remain an optional assistant without becoming geometry authority.

This direction does not portray Point.ai as discarded. Point.ai established the problem model, CAD vocabulary, review needs, and structured-geometry requirements. FloorplanFit preserves those lessons while choosing a more reliable source of truth for operational site-fit work.

### 13. Failure modes and safeguards

| Failure mode                       | System response or required safeguard                              |
|------------------------------------|--------------------------------------------------------------------|
| No walls detected                  | Fail quality gate and require review                               |
| Too few exterior walls             | Add explicit structural review reason                              |
| Weak shell coverage                | Mark footprint as insufficiently closed                            |
| No openings                        | Flag only when opening detection is expected                       |
| CubiCasa under-detects walls       | Fall back to heuristic walls                                       |
| Opening lacks compatible wall      | Filter it and record the reason                                    |
| Opening exceeds host span          | Reject it rather than clipping silently                            |
| Duplicate/furniture-like openings  | Filter through deterministic rules                                 |
| Model weights unavailable          | Return backend not ready                                           |
| Remote worker unavailable          | Return typed network/worker error                                  |
| Scale unverified                   | Preserve pixel unit and expose status                              |
| Reviewed annotations supplied      | Use them as generation override                                    |
| Artifact host replaced             | Temporary artifacts may be lost; use durable storage when required |
| Supabase not configured            | Disable persistence rather than faking a save                      |
| 3D assets unavailable or expensive | Keep disabled unless explicitly operationalized                    |
| Wildcard CORS in production        | Replace with an enforced allowlist                                 |
| Missing CAD template               | Fail readiness or use a verified fallback                          |

### 14. Extension strategy

Extensions should preserve the existing API while strengthening internal authority.

Recommended order:

- introduce typed internal geometry contracts;
- build a stable representative benchmark;
- construct a real wall graph;
- move opening anchoring from nearest-wall heuristics to graph edges;
- derive render geometry from graph faces and joins;
- upgrade the quality gate to explicit accept/review/reject semantics;
- add missing-opening creation to the review surface;
- move durable artifacts to managed storage;
- harden CORS, download authorization, backend readiness, and secrets handling;

- integrate optional AI only behind typed, validated contracts.

Avoid adding another model before the benchmark and geometry contract can measure whether it improves CAD validity.

## 15. Technology stack

### Runtime constraints

| Component        | Constraint     |
|------------------|----------------|
| Python container | 3.11           |
| FastAPI          | $\geq 0.135.0$ |
| Uvicorn          | $\geq 0.41.0$  |
| Pydantic         | $\geq 2.12.0$  |
| httpx            | $\geq 0.28.0$  |
| OpenCV headless  | $\geq 4.13.0$  |
| NumPy            | $\geq 2.3.0$   |
| Pillow           | $\geq 12.0.0$  |
| scikit-image     | $\geq 0.26.0$  |
| SciPy            | $\geq 1.17.0$  |
| Shapely          | $\geq 2.1.0$   |
| ezdxf            | 1.4.3          |
| React            | 19.x           |
| TypeScript       | 5.9.x          |
| Vite             | 8.x            |

ezdxf is pinned because CAD output behavior is sensitive to library changes. Several ML packages use lower-bound constraints rather than exact pins; reproducible deployments should lock the resolved environment after validation.

### Backend and AI

- Python;
- FastAPI and Uvicorn;
- Pydantic;
- httpx;
- OpenCV;
- NumPy, SciPy, scikit-image, Pillow;
- Shapely;
- PyTorch and torchvision;
- timm;
- segmentation-models-pytorch;
- Hugging Face Hub.

### CAD

- ezdxf;

- matplotlib rendering;
- AutoCAD MCP;
- AutoLISP/file IPC on Windows;
- optional headless CAD processing.

## Frontend

- React;
- TypeScript;
- Vite;
- Tailwind CSS;
- Vitest;
- Supabase JavaScript client;
- Three.js, React Three Fiber, and Drei for the disabled 3D surface.

## Persistence and hosting

- Supabase/PostgreSQL;
- Row Level Security;
- Vercel frontend hosting;
- Railway/Docker backend hosting;
- temporary filesystem storage for generated DXFs and artifacts.

---

# 16. Operator checklist

---

Before accepting a generated DXF:

- confirm the effective inference backend;
- confirm model weights and templates were ready;
- inspect `scale_status`;
- inspect wall, exterior-wall, and opening counts;
- read every review flag;
- inspect the structure and DXF previews;
- review door/window placement and direction;
- regenerate after edits;
- verify the downloaded DXF in the target CAD environment;
- persist the reviewed plan state;
- retain required artifacts in durable storage;
- do not promote the result to construction authority without the appropriate professional validation.

# Why Development Was Paused

## THE RESEARCH BOUNDARY

Point.ai reached a boundary where further progress depended less on routine implementation and more on structured-CAD research. The system already supported image-to-DXF processing, multiple inference paths, deterministic cleanup, human review, CAD generation, and operational application surfaces. The remaining question was the amount of proprietary vector ground truth, topology work, and validation required before the complete editable-CAD contract could be demonstrated responsibly.

### 1. Problems identified

- Domain shift: European-style CubiCasa plans and Pointe Homes drawings represented walls, dimensions, furniture, rooms, and visual conventions differently.
- Mixed training styles: combining sources often reduced real-world quality even when aggregate metrics appeared favorable.
- Corrupted inputs: a data-copy defect introduced black images into multiple runs. Correcting it removed one defect but did not remove the underlying style gap.
- Metric divergence: mask scores could improve while the visible result, UI review, or downstream CAD structure became worse.
- Structural inconsistency: wall and opening outputs could use incompatible coordinates, produce short gaps or missing branches, and leave openings floating or attached to the wrong wall.

### 2. Why raster-to-vector became research

A segmentation mask identifies pixels that probably belong to a wall. It does not define an authoritative CAD structure. It cannot guarantee exact endpoint coincidence, shared junction identity, closed room loops, consistent wall thickness, exterior-shell continuity, or ownership of each door and window by a specific wall. Small raster errors become material when the result must remain coherent, editable, and reusable in AutoCAD.

### 3. What responsible continuation required

Closing the gap required a broader architecture rather than another isolated training run:

- separate wall and opening models with a shared coordinate contract;
- a typed wall graph with corners, L/T/X junctions, connected components, exterior-shell identity, and thickness;
- explicit opening host edges, offsets, spans, side, orientation, and swing;
- a global geometric/topological solver for closure, continuity, merging, overshoot control, and the simplest valid plan;
- an exact DXF renderer for stable wall faces, cuts, trims, caps, layers, and editable joins;
- a CAD validator for connectivity, shell closure, widths, opening attachment, reverse-raster agreement, and target-environment editing;
- a representative benchmark plus roughly 200-500 corrected proprietary vector plans containing wall edges, junctions, widths, shell membership, attached openings, and verified CAD truth.

### 4. Why the demo threshold changed

A visually attractive preview or a DXF that merely opened was not enough. A responsible demo needed to prove that the same structural interpretation survived inference, normalization, topology reconstruction, opening ownership, exact rendering, and AutoCAD review without silently inventing or losing geometry. Reaching that threshold remained possible, but it had become a materially larger research and engineering program before a trustworthy demonstration could be claimed.

### 5. Decision and current status

The research horizon and delivery tradeoff were discussed with Carlos before work continued on FloorplanFit. FloorplanFit reduced immediate uncertainty by starting from architect-authored DXF geometry, making human-reviewed curation canonical, and applying deterministic changes only inside explicitly authorized zones.

Point.ai remains documented through its code, findings, failure modes, data requirements, and continuation architecture. Development can resume when the proprietary dataset, validation program, and research horizon justify reopening the raster-to-CAD path. The current pause keeps the remaining obligations explicit and prevents a plausible image from being presented as stronger CAD truth than the evidence supports.

**CURRENT STATUS: DOCUMENTED RESEARCH PATH**